

TrendSurfer Pro — Versione Minimal Sweep RR (Pine Script v6)

Strategia basata sul motore Sweep RR System: logica e filtri identici all'originale (Breakeven, Trailing Stop, un trade alla volta) — cambia solo la visualizzazione: nessuna tabella, linea, box o etichetta.

▲ **Triangolo verde**
▼ **Triangolo rosso**
× **Crocetta verde**
× **Crocetta rossa**

Punto d'ingresso LONG — apertura trade su Liquidity Sweep rialzista (tutti i filtri mantenuti)
Punto d'ingresso SHORT — apertura trade su Liquidity Sweep ribassista (tutti i filtri mantenuti)
Punto d'uscita dei LONG — TP / SL / Breakeven / Trailing Stop colpito
Punto d'uscita degli SHORT — TP / SL / Breakeven / Trailing Stop colpito

```
1 //@version=6
2 indicator(title = 'TrendSurfer Pro Sweep RR Minimal ", group = "Timezone")
22 utcOffset = input.int(2, "UTC Offset (hours)", minval = -12, maxval = 14, group = "Timezone", tooltip = "Set your timezone offset from UTC. Auto-
adjusts when changing assets - Imposta l'offset del fuso orario rispetto all'UTC.")
23
24 // === ATR VOLATILITY SCORE ===
25 atrVolPeriod = input.int(14, "ATR Period (Volatility Score)", minval = 1, group = "ATR Volatility Score ")
26 atrVolMAPeriod = input.int(5, "ATR MA Period (score calibration)", minval = 1, group = "ATR Volatility Score ", tooltip = "Moving average of the
ATR used as a reference. ATR > MA = high volatility, ATR < MA = low volatility")
27 atrVolSensitivity = input.float(2.0, "Sensitivity (1-5)", minval = 0.5, maxval = 5.0, step = 0.5, group = "ATR Volatility Score ", tooltip =
"Amplifies or reduces the score's response. Higher values = more responsive")
28 enableVolScoreFilter = input.bool(false, "Enable Volatility Score Master Filter ", group = "ATR Volatility Score ", tooltip = "Show signals
only if ATR Volatility Score >= minimum threshold")
29 volScoreMinThreshold = input.int(80, "Min Volatility Score (1-100)", minval = 1, maxval = 100, group = "ATR Volatility Score )
30
31 atrVol = ta.atr(atrVolPeriod)
32 atrVolMAAdapted =
33     timeframe.in_seconds() <= 60 ? atrVolMAPeriod * 5 :
34     timeframe.in_seconds() <= 120 ? atrVolMAPeriod * 3 :
35     timeframe.in_seconds() <= 180 ? atrVolMAPeriod * 2 :
36     timeframe.in_seconds() <= 300 ? atrVolMAPeriod :
37     timeframe.in_seconds() <= 900 ? math.round(atrVolMAPeriod / 2) :
38     timeframe.in_seconds() <= 1800 ? math.round(atrVolMAPeriod / 3) :
39     timeframe.in_seconds() <= 3600 ? math.round(atrVolMAPeriod / 5) :
40     timeframe.in_seconds() <= 14400 ? math.round(atrVolMAPeriod * 3) :
41     atrVolMAPeriod
42 atrVolMA = ta.sma(atrVol, math.max(2, atrVolMAAdapted))
43 atrVolDev = atrVolMA > 0 ? (atrVol / atrVolMA - 1.0) * 100.0 * atrVolSensitivity : 0.0
44 atrVolScore = math.max(1, math.min(100, math.round(50 + atrVolDev)))
45
46 // === ATR MODE ===
47 enableATRMode = input.bool(false, title = 'Enable ATR Mode ', group = "ATR Bands Filter", tooltip = "When enabled, signals are filtered: only
signals within the ATR channel around EMA750 are shown.")
48 atrMultiplier = input.float(5, title = 'ATR Channel Multiplier (1-100)', minval = 1, maxval = 100, step = 0.5, group = "ATR Bands Filter")
49
50 // === LIQUIDITY SWEEP (INGRESSI) ===
51 enableLiqSweep = input.bool(true, "Enable Liquidity Sweeps ", group = "Liquidity Sweep )
52 sweepLookback_TS = input.int(1, "Sweep Lookback (re-entry candles)", group = "Liquidity Sweep ", minval = 1, maxval = 10, tooltip = "Maximum
number of candles after a spike beyond a swing level to confirm price has re-entered the range.")
53 swingLenSweep_TS = input.int(5, "Swing Length - Sweep", group = "Liquidity Sweep ", minval = 1, maxval = 50, tooltip = "Number of candles left &
right used to identify swing highs/lows for Liquidity Sweep detection.")
54 sweepEMAFilter_TS = input.bool(true, "Apply EMA750 Filter to Sweeps", group = "Liquidity Sweep ", tooltip = "Bullish sweeps only above EMA750,
bearish only below")
55 sweepVIDYAFilter_TS = input.bool(false, "Apply VIDYA Filter to Sweeps", group = "Liquidity Sweep ", tooltip = "Bullish sweeps only above VIDYA,
bearish only below")
56 sweepVWAPFilter_TS = input.bool(false, "Apply VWAP Filter to Sweeps", group = "Liquidity Sweep )
57
58 // === SWEEP RR SYSTEM (GESTIONE TRADE E USCITE) ===
59 grpSweepRR = "Sweep RR System 
60 enableSweepTrades = input.bool(true, "Enable Sweep RR Trades ", group = grpSweepRR, tooltip = "Attiva il motore di trade: ingressi sugli sweep,
uscite su TP/SL/BE/Trailing. (Nell'originale era 'Show Trade on Chart', qui è il cuore dell'indicatore quindi è attivo di default.)")
61 useCustomRR_SW = input.bool(false, "Custom RR Mode  (ignora swing, usa RR fisso)", group = grpSweepRR, tooltip = "When enabled, TP is calculated
using a fixed Risk:Reward ratio instead of the previous swing level.")
62 rrSLPips_SW = input.float(500.0, "SL Buffer (pips oltre wick)", minval = 0.1, group = grpSweepRR, tooltip = "Extra pips added beyond the candle
wick to place the Stop Loss. Increase to reduce noise-triggered stops.")
63 rrRatio_SW = input.float(5.0, "Risk:Reward Ratio", minval = 0.1, step = 0.1, group = grpSweepRR, tooltip = "Used in Custom RR Mode: TP = entry ±
(SL distance × ratio). In standard mode the previous swing level is used as TP.")
64 enableBE_SW = input.bool(true, "Enable Breakeven ", group = grpSweepRR, tooltip = "When price reaches the Breakeven RR level, the Stop Loss is
automatically moved to entry price.")
65 beRatio_SW = input.float(2.0, "Breakeven at RR", minval = 0.1, step = 0.1, group = grpSweepRR, tooltip = "The Risk:Reward level at which Breakeven
is triggered. Example: 2.0 means BE activates when price moves 2x the SL distance in profit.")
```

```

66 enableTrailing_SW = input.bool(true, "Enable Trailing Stop 🛑", group = grpSweepRR)
67 trailStartRR_SW = input.float(3, "Trailing Stop Start at RR", minval = 0.1, step = 0.1, group = grpSweepRR, tooltip = "Defines at which Risk:Reward
level the trailing activates.")
68 trailStepPips_SW = input.float(500.0, "Trailing Stop Step (pips)", minval = 0.1, step = 0.1, group = grpSweepRR, tooltip = "Defines how many pips
the SL trails behind the price.")
69 overlapTrades_SW = input.bool(false, "Overlap Trades 🔄", group = grpSweepRR, tooltip = "Allow multiple trades to be open simultaneously. When
disabled, a new trade is only opened after the previous one closes.")
70
71 // === FILTRO ORARIO 1 ===
72 enableTimeFilter = input.bool(true, title = 'Enable Time Filter 1 ⌚', group = "Time Filter 1")
73 timeFilter1Session = input.session("0600-1400", title = 'Time Filter 1 Range', group = "Time Filter 1", tooltip = "Session format automatically
adjusts to asset timezone")
74 // === FILTRO ORARIO 2 ===
75 enableTimeFilter2 = input.bool(true, title = 'Enable Time Filter 2 ⌚', group = "Time Filter 2")
76 timeFilter2Session = input.session("1400-2200", title = 'Time Filter 2 Range', group = "Time Filter 2", tooltip = "Session format automatically
adjusts to asset timezone")
77 // === FILTRO ORARIO 3 ===
78 enableTimeFilter3 = input.bool(true, title = 'Enable Time Filter 3 ⌚', group = "Time Filter 3")
79 timeFilter3Session = input.session("2200-0600", title = 'Time Filter 3 Range', group = "Time Filter 3", tooltip = "Session format automatically
adjusts to asset timezone")
80
81 // === ALERTS SETTINGS ===
82 enableEntryLongAlert = input.bool(true, title = 'Enable Long Entry Alert ▲', group = "Alerts Settings 📢")
83 enableEntryShortAlert = input.bool(true, title = 'Enable Short Entry Alert ▼', group = "Alerts Settings 📢")
84 alertSweepTPSL = input.bool(true, "Alert Sweep TP/SL 📢", group = "Alerts Settings 📢", tooltip = "Send an alert when a Sweep trade hits TP, SL,
Breakeven or Trailing Stop.")
85
86 // =====
87 // FUNZIONI
88 // =====
89 getTimezoneString() =>
90     useUTC ? "UTC" + (utcOffset >= 0 ? "+" : "-") + str.toString(utcOffset) : str.toString(utcOffset) : ""
91
92 passesVolScoreFilter() =>
93     not enableVolScoreFilter or atrVolScore >= volScoreMinThreshold
94
95 vidya_calc(src_vidya, vidya_length, vidya_momentum) =>
96     float momentum = ta.change(src_vidya)
97     float sum_pos_momentum = math.sum(momentum >= 0 ? momentum : 0.0, vidya_momentum)
98     float sum_neg_momentum = math.sum(momentum <= 0 ? 0.0 : -momentum, vidya_momentum)
99     float abs_cmo = math.abs(100 * (sum_pos_momentum - sum_neg_momentum) / (sum_pos_momentum + sum_neg_momentum))
100     float alpha = 2 / (vidya_length + 1)
101     var float vidya_value = 0.0
102     vidya_value := alpha * abs_cmo / 100 * src_vidya + (1 - alpha * abs_cmo / 100) * nz(vidya_value[1])
103     ta.sma(vidya_value, 15)
104
105 // === TIME FILTER ===
106 isInTimeRange() =>
107     if not enableTimeFilter and not enableTimeFilter2 and not enableTimeFilter3
108         true
109     else
110         inRange1 = enableTimeFilter ? not na(time(timeframe.period, timeFilter1Session, getTimezoneString())) : false
111         inRange2 = enableTimeFilter2 ? not na(time(timeframe.period, timeFilter2Session, getTimezoneString())) : false
112         inRange3 = enableTimeFilter3 ? not na(time(timeframe.period, timeFilter3Session, getTimezoneString())) : false
113         inRange1 or inRange2 or inRange3
114
115 // =====
116 // TYPES
117 // =====
118 type SweepTrade
119     float slLevel
120     float tpLevel
121     float entryPrice
122     float origSL
123     bool isBull
124     bool beTriggered
125     bool trailActivated
126     float trailHighWater
127
128 var array<SweepTrade> activeSweepTrades = array.new<SweepTrade>()
129
130 // =====
131 // BASE INDICATORS
132 // =====
133 ema750 = ta.ema(close, 750)
134 smootherATR = ta.ema(ta.tr, 750)
135 upperBand = ema750 + atrMultiplier * smootherATR
136 lowerBand = ema750 - atrMultiplier * smootherATR
137 vwap = ta.vwap(hl2)
138 pipSize = syminfo.mintick * (syminfo.type == "forex" ? 10 : 1)
139
140 // VIDYA (parametri fissi come nell'originale)
141 vidya_length = 10
142 vidya_momentum = 20
143 vidya_value = vidya_calc(close, vidya_length, vidya_momentum)
144
145 // =====
146 // LIQUIDITY SWEEP (logica invariata)
147 // =====
148 swingHighSweepTS = ta.pivohigh(high, swingLenSweep_TS, swingLenSweep_TS)
149 swingLowSweepTS = ta.pivotlow(low, swingLenSweep_TS, swingLenSweep_TS)
150 var float lastSHSweepTS = na
151 var float lastSLSweepTS = na
152 var int lastSHBarSweepTS = na

```

```

153 var int lastSLBarSweepTS = na
154 var float prevSHSweepTS = na
155 var float prevSLSweepTS = na
156 if not na(swingHighSweepTS)
157     prevSHSweepTS := lastSHSweepTS
158     lastSHSweepTS := swingHighSweepTS
159     lastSHBarSweepTS := bar_index - swingLenSweep_TS
160 if not na(swingLowSweepTS)
161     prevSLSweepTS := lastSLSweepTS
162     lastSLSweepTS := swingLowSweepTS
163     lastSLBarSweepTS := bar_index - swingLenSweep_TS
164 bearSweep_TS1 = not na(lastSHSweepTS) and high > lastSHSweepTS and close < lastSHSweepTS
165 bullSweep_TS1 = not na(lastSLSweepTS) and low < lastSLSweepTS and close > lastSLSweepTS
166 bearSweep_TS2 = false
167 bullSweep_TS2 = false
168 if not na(lastSHSweepTS)
169     for i = 1 to sweepLookback_TS
170         if close[i] > lastSHSweepTS and close < lastSHSweepTS
171             bearSweep_TS2 := true
172             break
173 if not na(lastSLSweepTS)
174     for i = 1 to sweepLookback_TS
175         if close[i] < lastSLSweepTS and close > lastSLSweepTS
176             bullSweep_TS2 := true
177             break
178 bullSweep_TS = enableLiqSweep and (bullSweep_TS1 or bullSweep_TS2) and isInRange() and (not sweepEMAFilter_TS or close > ema750) and (not
sweepVIDYAFilter_TS or (not na(vidya_value) and close > vidya_value)) and (not sweepVWAPFilter_TS or close > vwap) and passesVolScoreFilter() and (not
enableATRMode or close <= upperBand)
179 bearSweep_TS = enableLiqSweep and (bearSweep_TS1 or bearSweep_TS2) and isInRange() and (not sweepEMAFilter_TS or close < ema750) and (not
sweepVIDYAFilter_TS or (not na(vidya_value) and close < vidya_value)) and (not sweepVWAPFilter_TS or close < vwap) and passesVolScoreFilter() and (not
enableATRMode or close >= lowerBand)
180
181 // =====
182 // AGGIORNA TRADE SWEEP ATTIVI (Breakeven → TP/SL → Trailing, invariato)
183 // =====
184 bool exitLongThisBar = false
185 bool exitShortThisBar = false
186 bool entryLongThisBar = false
187 bool entryShortThisBar = false
188
189 if array.size(activeSweepTrades) > 0
190     for i = array.size(activeSweepTrades) - 1 to 0
191         t = array.get(activeSweepTrades, i)
192         // Breakeven
193         if enableBE_SW and not t.beTriggered
194             sLDist = math.abs(t.entryPrice - t.origSL)
195             beLevel = t.isBull ? t.entryPrice + sLDist * beRatio_SW : t.entryPrice - sLDist * beRatio_SW
196             if (t.isBull ? high >= beLevel : low <= beLevel)
197                 t.sLLLevel := t.entryPrice
198                 t.beTriggered := true
199         // Uscita: TP o SL (incluso SL spostato da BE/Trailing)
200         sLHit = t.isBull ? low <= t.sLLLevel : high >= t.sLLLevel
201         tpHit = t.isBull ? high >= t.tpLevel : low <= t.tpLevel
202         if sLHit or tpHit
203             pips = math.abs(tpHit ? t.tpLevel - t.entryPrice : t.sLLLevel - t.entryPrice) / pipSize
204             isProfit = t.isBull ? t.sLLLevel > t.entryPrice : t.sLLLevel < t.entryPrice
205             isBE = t.sLLLevel == t.entryPrice
206             sLPipsForRR = math.abs(t.entryPrice - t.origSL) / pipSize
207             tradeRR = sLPipsForRR > 0 ? pips / sLPipsForRR : 0.0
208             if t.isBull
209                 exitLongThisBar := true
210             else
211                 exitShortThisBar := true
212             if alertSweepTPSL
213                 if tpHit
214                     alert("SWEEP TP HIT 🟢" + str.toString(pips, "#.#") + " pips / " + str.toString(tradeRR, "#.##") + "R",
alert.freq_once_per_bar)
215                 else if isProfit
216                     alert("SWEEP TRAIL HIT 🟢" + str.toString(pips, "#.#") + " pips / " + str.toString(tradeRR, "#.##") + "R",
alert.freq_once_per_bar)
217                 else if isBE
218                     alert("SWEEP BE HIT 🟡 0 pips", alert.freq_once_per_bar)
219                 else
220                     alert("SWEEP SL HIT 🔴" + str.toString(pips, "#.#") + " pips / " + str.toString(tradeRR, "#.##") + "R",
alert.freq_once_per_bar)
221             array.remove(activeSweepTrades, i)
222         else
223             // Trailing Stop
224             if enableTrailing_SW
225                 sLDist = math.abs(t.entryPrice - t.origSL)
226                 trailTrigger = t.isBull ? t.entryPrice + sLDist * trailStartRR_SW : t.entryPrice - sLDist * trailStartRR_SW
227                 trailStep = trailStepPips_SW * pipSize
228                 if not t.trailActivated
229                     if (t.isBull ? high >= trailTrigger : low <= trailTrigger)
230                         t.trailActivated := true
231                         t.trailHighWater := t.isBull ? high : low
232                 if t.trailActivated
233                     if t.isBull
234                         if high > t.trailHighWater
235                             t.trailHighWater := high
236                             newSL = t.trailHighWater - trailStep
237                             if newSL > t.sLLLevel
238                                 t.sLLLevel := newSL
239                     else

```

```

240         if low < t.trailHighWater
241             t.trailHighWater := low
242         newSL = t.trailHighWater + trailStep
243         if newSL < t.sllLevel
244             t.sllLevel := newSL
245
246 // =====
247 // APRI NUOVI TRADE SWEEP (INGRESSI, logica invariata)
248 // =====
249 sw_canLong = overlapTrades_SW or array.size(activeSweepTrades) == 0
250 sw_canShort = overlapTrades_SW or array.size(activeSweepTrades) == 0
251 if enableSweepTrades and bullSweep_TS and sw_canLong
252     entry = close
253     slDist = math.max((entry - low), pipSize) + rrSLPips_SW * pipSize
254     tpLvl = useCustomRR_SW
255         ? entry + slDist * rrRatio_SW
256         : (not na(prevSHSweepTS) and prevSHSweepTS > entry ? prevSHSweepTS : entry + slDist * rrRatio_SW)
257     array.push(activeSweepTrades, SweepTrade.new(entry - slDist, tpLvl, entry, entry - slDist, true, false, false, float(na)))
258     entryLongThisBar := true
259 if enableSweepTrades and bearSweep_TS and sw_canShort
260     entry = close
261     slDist = math.max((high - entry), pipSize) + rrSLPips_SW * pipSize
262     tpLvl = useCustomRR_SW
263         ? entry - slDist * rrRatio_SW
264         : (not na(prevSLSweepTS) and prevSLSweepTS < entry ? prevSLSweepTS : entry - slDist * rrRatio_SW)
265     array.push(activeSweepTrades, SweepTrade.new(entry + slDist, tpLvl, entry, entry + slDist, false, false, false, float(na)))
266     entryShortThisBar := true
267
268 // =====
269 // VISUALIZZAZIONE - SOLO I 4 PUNTI RICHIESTI
270 // =====
271 plotshape(entryLongThisBar, title = 'Ingresso Long ▲', location = location.belowbar, color = color.new(color.green, 0), style = shape.triangleup,
size = size.small)
272 plotshape(entryShortThisBar, title = 'Ingresso Short ▼', location = location.abovebar, color = color.new(color.red, 0), style = shape.triangledown,
size = size.small)
273 plotshape(exitLongThisBar, title = 'Uscita Long ×', location = location.abovebar, color = color.new(color.green, 0), style = shape.xcross, size =
size.small)
274 plotshape(exitShortThisBar, title = 'Uscita Short ×', location = location.belowbar, color = color.new(color.red, 0), style = shape.xcross, size =
size.small)
275
276 // =====
277 // ALERTS
278 // =====
279 alertcondition(entryLongThisBar, title = 'Ingresso Long ▲', message = '▲ LONG ENTRY (Bullish Liquidity Sweep)')
280 alertcondition(entryShortThisBar, title = 'Ingresso Short ▼', message = '▼ SHORT ENTRY (Bearish Liquidity Sweep)')
281 alertcondition(exitLongThisBar, title = 'Uscita Long ×', message = '× LONG EXIT (TP/SL/BE/Trailing)')
282 alertcondition(exitShortThisBar, title = 'Uscita Short ×', message = '× SHORT EXIT (TP/SL/BE/Trailing)')
283 if entryLongThisBar and enableEntryLongAlert
284     alert('▲ LONG ENTRY (Bullish Liquidity Sweep) at ' + str.toString(close, format.mintick), alert.freq_once_per_bar_close)
285 if entryShortThisBar and enableEntryShortAlert
286     alert('▼ SHORT ENTRY (Bearish Liquidity Sweep) at ' + str.toString(close, format.mintick), alert.freq_once_per_bar_close)
287

```